



NORTH SOUTH UNIVERSITY

School of Engineering & Physical Sciences

**Project: 23-Bit Single Cycle MIPS CPU with
R-type, I-type, and J-type Instructions**

Group Number: 04

Submitted To:

Department of Electrical & Computer Engineering (ECE)

Course Instructor: Tanjila Farah (TnF)

Course: Computer Organization & Architecture

Course ID: CSE 332

Section: 04

Semester: Summer 2024

Submitted By:

Group Member's Name	ID
Moontaha Rawshan	2221035642
Taslia Zaman	2211007642

Design of a 23-Bit Single Cycle MIPS CPU

Objective: Designing a 23-bit MIPS CPU based on ISA with particular operations following R-type, I-type, J-type format.

1. How many Operands?

There are Five operands. Represented as,

Rd = Destination Register

Rs = Source Register

Rt = Temporary Register(Source Register2)

Shamt = Shift Amount

Funct = Function

2. Type of Operands?

- a) Register-based: Operands are stored in registers.
- b) Memory-based: Operands are fetched from memory.

3. How many Operations?

There are 7 types of operations.

4. Types of Operations?

Categories of 7 type of operations are:

- a) Arithmetic Operations
- b) Logical Operations
- c) Conditional /Unconditional Operations
- d) Shift Operations
- e) Memory Operations
- f) Comparison Operations
- g) No Operation

5. No. of the format of instruction (how many different formats?)

We need 3 formats for our 23-bit MIPS CPU based on ISA. These are:

- i. Register Type (R-Type)
- ii. Immediate Type (I-Type)
- iii. Jump Type (J-Type)

6. Describe each of the formats (fields and field length)

Here 3 formats with fields and field length shown below-

R-Type ISA FORMAT:

OPCODE	RS	RT	RD	SHAMT	FUNCT
3	5	5	5	2	3

I-Type ISA FORMAT:

OPCODE	RS	RT	IMMEDIATE
3	5	5	10

J-Type ISA FORMAT:

OPCODE	ADDRESS
3	20

Opcode Assignment Table:

Instruction Category	Instruction	Operation	Opcode	Type
COMPARISON	SLT	Shift less than	000	R
SHIFT	SLL	Shift left logical	000	R
ARITHMETIC	SUB	Substract	000	R
UNCONDITIONAL	JMP	Jump	001	J
LOGICAL	AND	And	000	R
MEMORY	LW	Load	010	I
ARITHMETIC	ADDi	Add immediate	011	I
-	NOP	No operation	100	-
LOGICAL	NOR	Nor	000	R
CONDITIONAL	Beq	Branch on Equal	101	I
LOGICAL	OR	Or	000	R
ARITHMETIC	ADD	Add	000	R
CONDITIONAL	BNE	Branch on Not Equal	110	I
SHIFT	SRL	Shift Right Logical	000	R
MEMORY	SW	Store	111	I

Control Signal Definitions:

RegDst:

- I. 1 for R-type (destination = RD)
- II. 0 for I-type (destination = RT)
- III. X for J-type (unused)

ALUSrc:

- I. 0 for register-based ALU inputs (R-type)
- II. 1 for immediate values (I-type)

MemRead:

- I. 1 for LW
- II. 0 otherwise

MemWrite:

- I. 1 for SW
- II. 0 otherwise

MemToReg:

- I. 1 for LW (data from memory to register)
- II. 0 for ALU results

RegWrite:

- I. 1 for instructions that write to a register (R-type, I-type except SW)
- II. 0 for SW, BEQ, BNE, JMP, and NOP

The ALUOp field directs the ALU's operation by encoding the type of operation the ALU performs. In R-type instructions, the **FUNCT field** in the instruction further specifies the exact operation (e.g., ADD, SUB, SLL). For I-type instructions like ADDi, the opcode itself determines the operation.

Control Unit Table:

Instruction	Opco de	Alu Op	Fun ct	Alu Src	Reg Dest	Mem red	Mem write	Mem To Reg	Reg write	Branch	Jump
SLT	000	10	000	0	1	0	0	0	1	0	0
SLL	000	10	001	0	1	0	0	0	1	0	0
SUB	000	10	010	0	1	0	0	0	1	0	0
JMP	001	XX	XXX	X	X	0	0	0	0	0	1
AND	000	10	011	0	1	0	0	0	1	0	0
LW	010	01	XXX	1	0	1	0	1	1	0	0
ADDi	011	01	XXX	1	0	0	0	0	1	0	0
NOP	100	XX	XXX	X	X	0	0	0	0	0	0
NOR	000	10	100	0	1	0	0	0	1	0	0
BEQ	101	00	XXX	0	X	0	0	0	0	1	0
OR	000	10	101	0	1	0	0	0	1	0	0
ADD	000	10	110	0	1	0	0	0	1	0	0
BNE	110	00	XXX	0	X	0	0	0	0	1	0
SRL	000	10	111	0	1	0	0	0	1	0	0
SW	111	01	XXX	1	X	0	1	0	0	0	0

Encoding for ALUOp:

- 00: BRANCHING
- 01: I -TYPE
- 10: R -TYPE

Translations of high-level codes into machine code:

SOME EXAMPLES :

High Level Code	Assembly Code	Comment
$p = p + q$	add \$r1 \$r1 \$r2	\$r1 = p \$r2 = q \$r1 gets \$r1 + \$r2
$a = a + 5$	addi \$r1 \$r1 5	\$r1 = a \$r1 gets \$r1 + 5
$p = p - q$	sub \$r1 \$r1 \$r2	\$r1 = p \$r2 = q \$r1 gets \$r1 - \$r2
$a = a \text{ AND } b$	sub \$r1 \$r1 \$r2	\$r1 = a \$r2 = b \$r1 gets \$r1 AND \$r2
<pre>b = 0; for (i = 0; i < 2; i++) { b = b + 6; }</pre>	<pre>sub \$r0 \$r0 \$r0 sub \$r1 \$r1 \$r1 sub \$r2 \$r2 \$r2 addi \$r2 \$r2 2 beq \$r1 \$r2 3 addi \$r0 \$r0 4 addi \$r1 \$r1 1 jmp 4 sub \$r15 \$r15 \$r15 sw \$r0 \$r15 15 sub \$r15 \$r15 \$r15 lw \$r12 \$r15 15</pre>	<pre>beq: (4 - 1) = 3 [beq \$r1 \$r2 3] jmp: (4) From, [jmp 4] To, [beq \$r1 \$r2 3]</pre>

Why Using 3 Bits for Opcode?

Even with 15 operations, the design works because:

1. Opcode Groups Instruction Types:

- Only I-Type, Branch, and Jump instructions require distinct opcodes.
- All R-Type instructions share a single opcode (000) and rely on the function field for further differentiation.

2. 3 Bits Are Efficient:

- Using 3 bits for the opcode keeps the instruction format compact.
- A 4-bit opcode (16 total codes) would waste bits since we only have 15 operations.

3. Function Field Extends Opcodes:

- By combining the opcode and function field, we can efficiently support all 15 operations.

The 3-bit opcode in our 23-bit MIPS architecture works because:

1. R-Type instructions share one opcode (000), with differentiation in the 3-bit function field.
2. Other instruction types (I-Type, Branch, Jump) each get unique opcodes.
3. This design is compact, efficient, and avoids wasting bits while accommodating all 15 operations.

Limitations:

- In this 23-bit CPU, only 15 operations were used and in maximum 23 operations can be used.
- This CPU doesn't have any error handling functionalities and cannot address any sort of hazards; thus the user must validate instructions in the correct order that is supported by the CPU.